Clean Code

Praktiska exempel från Labb 2

Magic Numbers & Förklarande variabler

Varför? Siffran 5 är ett klassiskt "Magic Number" som dyker upp från ingenstans. Genom att lägga det i en väl namngiven variabel (VisionRangeRadius) förstår läsaren direkt varför talet 5 används, och formeln blir plötsligt självförklarande.

Gör INTE:

```
if (Math.Sqrt(Math.Pow(x1 - x2, 2) + Math.Pow(y1 - y2, 2)) <= 5)
{
  isVisible = true;
}
```

Gör HELLRE:

```
double distanceToPlayer = Math.Sqrt(Math.Pow(x1 - x2, 2) + Math.Pow(y1 - y2, 2));
int visionRangeRadius = 5;

if (distanceToPlayer <= visionRangeRadius)
{
  isVisible = true;
}
```

Magic Numbers & Enum

•**Varför?** Siffrorna 0, 1, 2, 3 säger ingenting om *vilket* håll karaktären faktiskt rör sig på. Om man råkar skriva fel (t.ex. blandar ihop 0 och 1) uppstår dolda buggar. En enum ger koden ett semantiskt värde – den dokumenterar sig själv.

Gör INTE:

```
1  int direction = _rnd.Next(4);
2
3  switch (direction)
4  {
5    case 0: targetY--; break;
6    case 1: targetY++; break;
7    case 2: targetX--; break;
8    case 3: targetX++; break;
9  }
```

Gör HELLRE:

```
1  public enum Direction { North, South, East, West }
2
3  // ...
4
5  Direction direction = (Direction)_rnd.Next(4);
6
7  switch (direction)
8  {
9    case Direction.North: targetY--; break;
10   case Direction.South: targetY++; break;
11   case Direction.West: targetX--; break;
12   case Direction.East: targetX++; break;
13 }
```

Självdokumenterande kod: Named Arguments

Varför? Dessa siffror är inte "magiska" i sig eftersom de skickas till en konstruktor som konfigurerar objektet. Men utan namnen är det omöjligt att veta vilken siffra som gör vad. Genom att använda parameterNamn: blir anropet självdokumenterande.

Gör INTE:

```
1  AttackDice = new Dice(2, 6, 2);
```

Gör HELLRE:

```
1  AttackDice = new Dice(numberOfDice: 2, sidesPerDice: 6, modifier: 2);
```

DRY-principen (Don't Repeat Yourself)

Gör INTE:

```
public void ResolveCombat(Player player, Enemy enemy)
{
    // Spelaren attackerar monstret
    int playerAttack = player.AttackDice.Throw();
    int enemyDefence = enemy.DefenceDice.Throw();
    int damageToEnemy = playerAttack - enemyDefence;

    if (damageToEnemy > 0)
    {
        enemy.HP -= damageToEnemy;
        Console.WriteLine($"{player.Name} gör {damageToEnem
    }

    if (enemy.HP > 0)
    {
        int enemyAttack = enemy.AttackDice.Throw();
        int playerDefence = player.DefenceDice.Throw();
        int damageToPlayer = enemyAttack - playerDefence;

        if (damageToPlayer > 0)
        {
            player.HP -= damageToPlayer;
            Console.WriteLine($"{enemy.Name} gör {damageToP
        }
    }
}
```

- I "Gör INTE"-exemplet är logiken för tärningskast, skadeberäkning, HP-avdrag och textutskrifter helt identisk på två ställen. Om du vill ändra stridsreglerna i framtiden (t.ex. lägga till kritisk skada) måste du komma ihåg att ändra på båda ställena.

- I "Gör HELLRE"-exemplet utnyttjar vi att både Player och Enemy delar egenskaper (via en gemensam bastyp som Creature eller LevelEntity). Hela anfallet blir en enda metod som vi bara anropar två gånger. Mindre kod betyder färre ställen för buggar att gömma sig på!

Gör HELLRE:

```
public void ResolveCombat(Player player, Enemy enemy)
{
    // Spelaren slår först
    ExecuteAttack(attacker: player, defender: enemy);

    // Om monstret överlever, slår det tillbaka med exakt samma metod!
    if (enemy.HP > 0)
    {
        ExecuteAttack(attacker: enemy, defender: player);
    }
}

// Den utbrutna, återanvändbara metoden (tar emot basklassen/interfacet Creature/Entity)
private void ExecuteAttack(Creature attacker, Creature defender)
{
    int attackScore = attacker.AttackDice.Throw();
    int defenceScore = defender.DefenceDice.Throw();
    int damage = attackScore - defenceScore;

    if (damage > 0)
    {
        defender.HP -= damage;
        Console.WriteLine($"{attacker.Name} gör {damage} skada på {defender.Name}!");
    }
}
```

DRY i kontroll-strukturer

Gör INTE:

```
if (input == ConsoleKey.UpArrow)
{
    Position nextPos = new Position(player.X, player.Y - 1);
    if (myLevel.CanWalkTo(nextPos))
    {
        player.Y--;
        myLevel.UpdateScreen();
    }
}
else if (input == ConsoleKey.DownArrow)
{
    Position nextPos = new Position(player.X, player.Y + 1);
    if (myLevel.CanWalkTo(nextPos))
    {
        player.Y++;
        myLevel.UpdateScreen();
    }
}
else if (input == ConsoleKey.LeftArrow)
{
    Position nextPos = new Position(player.X - 1, player.Y);
    if (myLevel.CanWalkTo(nextPos))
    {
        player.X--;
        myLevel.UpdateScreen();
    }
}
else if (input == ConsoleKey.RightArrow)
{
    Position nextPos = new Position(player.X + 1, player.Y);
    if (myLevel.CanWalkTo(nextPos))
    {
        player.X++;
        myLevel.UpdateScreen();
    }
```

•I "Gör INTE"-exemplet skapas en massiv kodvägg där samma logiska struktur upprepas fyra gånger, trots att det bara är en koordinat och ett tecken som skiljer dem åt. Detta är svårläst och skapar fyra gånger så stor yta för slarvfel (t.ex. att råka skriva player.X++ under LeftArrow).

•I "Gör HELLRE"-exemplet separerar vi *avsikten* (vilket håll vill jag gå?) från *handlingen* (får jag gå dit och hur ritas det ut?). Switchen blir en ren och snabbläst "översättare", medan den faktiska spelregeln bara underhålls på ett enda ställe.

Gör HELLRE:

```
// 1. Ta reda på den tänkta förändringen (Delta X och Y)
int deltaX = 0;
int deltaY = 0;

switch (input)
{
    case ConsoleKey.UpArrow:    deltaY = -1; break;
    case ConsoleKey.DownArrow:  deltaY =  1; break;
    case ConsoleKey.LeftArrow:  deltaX = -1; break;
    case ConsoleKey.RightArrow: deltaX =  1; break;
}

// 2. Kör den gemensamma logiken EN gång under switchen
if (deltaX != 0 || deltaY != 0)
{
    Position nextPos = new Position(player.X + deltaX, player.Y + deltaY);

    if (myLevel.CanWalkTo(nextPos))
    {
        player.X += deltaX;
        player.Y += deltaY;
        myLevel.UpdateScreen();
    }
}
```

Guard Clauses (Tidiga returner)

Gör INTE:

```
public bool TryMove(Position newPosition)
{
    if (IsWithinBounds(newPosition))
    {
        LevelElement element = GetElementAt(newPosition);

        if (element == null)
        {
            this.Position = newPosition;
            return true;
        }
        else
        {
            if (element is Enemy)
            {
                StartCombat((Enemy)element);
                return true;
            }
            else
            {
                return false;
            }
        }
    }
    else
    {
        return false;
    }
}
```

- I "Gör INTE"-exemplet tvingas läsaren hålla fem olika if och else-block i huvudet samtidigt för att förstå när metoden faktiskt returnerar true eller false.

- I "Gör HELLRE"-exemplet vänder vi på logiken. Vi kollar efter felaktigheter eller specialfall *direkt* längst upp i metoden och "avbryter" (return) direkt om villkoret uppfylls. Det gör att resten av metoden blir helt platt, lättläst och saknar else-block.

Gör HELLRE:

```
public bool TryMove(Position newPosition)
{
    // Guard Clause 1: Kolla om vi går utanför kartan
    if (!IsWithinBounds(newPosition)) return false;

    LevelElement element = GetElementAt(newPosition);

    // Guard Clause 2: Om vägen är helt tom, flytta direkt
    if (element == null)
    {
        this.Position = newPosition;
        return true;
    }

    // Guard Clause 3: Om det är en fiende, starta strid
    if (element is Enemy enemy)
    {
        StartCombat(enemy);
        return true;
    }

    // Standardfall: Vi krockade med något annat (t.ex. en vägg)
    return false;
}
```

Abstraktionsnivå
(Håll detaljerna borta från ytan)

Gör INTE:

```
while (!gameover)
{
    var key = Console.ReadKey(true).Key;

    // Detaljer för spelarrörelse insprängt direkt i loopen
    int nextX = player.X;
    int nextY = player.Y;
    if (key == ConsoleKey.UpArrow) nextY--;
    // ... fler ifs ...

    // Detaljer för kollision insprängt
    bool canMove = true;
    foreach (var entity in elements)
    {
        if (entity.X == nextX && entity.Y == nextY && !entity.IsPassable)
            canMove = false;
    }

    if (canMove) { player.X = nextX; player.Y = nextY; }

    // Detaljer för fiende-uppdatering och rendering om vartannat...
    foreach (var enemy in elements.OfType<Enemy>())
    {
        // Slumpmässig rörelse för råttor, etc. direkt här i loopen...
    }

    Console.Clear();
    // Massor av rader för att rita kartan...
}
```

• I "Gör INTE"-exemplet blandas hög nivå (är spelet slut?) med extremt låg nivå (vilken pixel/koordinat ändras?). Läsaren tvingas gräva ner sig i matematiken för att ens förstå att koden försöker flytta en karaktär.

•I "Gör HELLRE"-exemplet döljer vi implementationsdetaljerna inuti väl namngivna metoder. Spelloopen blir helt självdokumenterande. Du kan på två sekunder läsa exakt i vilken ordning spelet körs, och om du vill veta hur renderingen går till, så klickar du dig bara in i RenderGame.

Gör HELLRE:

```
// Huvudloopen är nu en ren innehållsförteckning
// på hög abstraktionsnivå
while (!gameover)
{
    ConsoleKey input = GetUserInput();

    UpdatePlayer(input, levelData);

    UpdateEnemies(levelData);

    RenderGame(levelData);

    if (player.IsDead) gameover = true;
}
```

Principle of Least Surprise
(Lura inte läsaren)

Varför? En metod som börjar med *Get*, *Check* eller *Is* förväntas vara "säker" att anropa utan att spelets tillstånd muteras (ändras). Om en utvecklare anropar GetElementAt bara för att titta om en ruta är tom, och ett monster plötsligt raderas från spelet, skapas extremt svårhittade buggar. Namnet på en metod måste vara ett ärligt löfte om vad som händer inuti.

Gör INTE:

```
public LevelElement GetElementAt(Position position)
{
    foreach (var element in elements)
    {
        if (element.Position == position)
        {
            elements.Remove(element);
            return element;
        }
    }
    return null;
}
```

Gör HELLRE:

```
// 1. Renodlad "Get"-metod utan dolda sidosystem
public LevelElement GetElementAt(Position position)
{
    return elements.FirstOrDefault(e => e.Position == position);
}

// 2. Om du faktiskt vill plocka bort något,
//    gör det till en egen tydlig handling
public void RemoveElement(LevelElement element)
{
    elements.Remove(element);
}
```

Encapsulation (Tell, Don't Ask)

• Istället för att omvärlden ska behöva fråga objektet om dess data ("Vad har du för HP?"), göra beräkningar utifrån, och sedan tvinga på objektet ett nytt värde, så ska vi bara ge objektet en order ("Ta skada!").

• Det är detta som är äkta inkapsling: objektet äger och skyddar sitt eget tillstånd, medan omvärlden bara behöver bry sig om vilka metoder som går att anropa.

Gör INTE:

```
// Klassen Enemy är bara en "dum"
// databehållare utan eget beteende.
public class Enemy : LevelElement
{
    public int HP { get; set; }
    public string Name { get; set; }
}

// ... Någonstans mitt i stridslogiken:
if (damage > 0)
{
    // Spelloopen går in och opererar direkt
    // på monstrets inre organ (data).
    enemy.HP = enemy.HP - damage;

    if (enemy.HP <= 0)
    {
        myLevel.Elements.Remove(enemy);
        Console.WriteLine($"{enemy.Name} dog!");
    }
```

Gör HELLRE:

```
public class Enemy : LevelElement
{
    // HP kan läsas utifrån,
    // men bara ändras inifrån.
    public int HP { get; private set; }
    public string Name { get; private set; }

    public void TakeDamage(int amount)
    {
        // Defensiv programmering
        // (metoden skyddar sig själv)
        if (amount <= 0) return;

        HP -= amount;
        if (HP < 0) HP = 0;
    }
}

// ... Någonstans mitt i stridslogiken:
enemy.TakeDamage(damage);

// ... Efter att alla drag är klara städas
// spelvärlden upp på ett centralt ställe:
RemoveAllEnemiesWithZeroHp();
```

• I "Gör INTE"-exemplet måste rörelselogiken ha stenkoll på alla dussintals element som kan finnas i spelet och veta exakt hur man hanterar dem. Det bryter mot Open/Closed-principen – varje gång en ny typ av objekt läggs till i spelet måste den centrala förflyttningsmetoden skrivas om.

• I "Gör HELLRE"-exemplet vänder vi på ansvaret. Rörelselogiken frågar bara elementet på målrutan: "Spelaren försöker gå på dig, vad händer?". Väggen svarar false och stoppar spelaren, medan monstret startar en strid. Koden blir platt, flexibel och helt fri från is-kontroller.

Gör INTE:

```
// Inuti förflyttningslogiken när vi kontrollerar målrutan
LevelElement elementAtTarget = levelData.GetElementAt(newX, newY);

if (elementAtTarget != null)
{
    if (elementAtTarget is Wall)
    {
        return; // Stopp, kan inte gå genom väggar
    }
    else if (elementAtTarget is Enemy)
    {
        // Starta strid om det är en fiende
        StartCombat((Enemy)elementAtTarget);
        return;
    }
    // Om vi lägger till Loot, Fällor eller NPC:er så kommer
    // vi också behöva lägga till ännu fler if-satser här!
}
```

Polymorfism vid interaktioner (Tell, Don't Ask)

Gör HELLRE:

```
// 1. I basklassen LevelElement definieras en virtuell metod för krockar:
public abstract class LevelElement
{
    // Standard: Går att gå på
    public virtual bool HandleCollision(Player player) => true;
}

// 2. Subklasserna skriver över beteendet utifrån sina egna regler:
public class Wall : LevelElement
{
    // Väggar stoppar spelaren
    public override bool HandleCollision(Player player) => false;
}

public class Enemy : LevelElement
{
    public override bool HandleCollision(Player player)
    {
        StartCombat(this); // Fiender startar strid
        return false;
    }
}

// 3. I spel-loopen behövs nu inga typkontroller eller if-satser alls:
LevelElement target = levelData.GetElementAt(newX, newY);

if (target == null || target.HandleCollision(player))
{
    player.MoveTo(newX, newY);
}
```

Hög Sammanhållning / Cohesion (Var sak på sin plats)

Gör INTE:

```
public abstract class LevelElement
{
    public int X { get; set; }
    public int Y { get; set; }

    public static double GetDistanceBetween(int x1, int y1, int x2, int y2)
    {
        return Math.Sqrt(Math.Pow(x1 - x2, 2) + Math.Pow(y1 - y2, 2));
    }
}

// Exempel inuti spel-loopen:
double d = LevelElement.GetDistanceBetw
```

I "Gör INTE"-exemplet tar metoden bara emot lösa heltal. Den har ingen koppling till det faktiska objektet. Du skulle kunna anropa metoden via en *Wall* för och räkna ut avståndet mellan två monster. Det bryter mot idén om att hålla ihop data och funktioner.

Gör HELLRE:

```
public abstract class LevelElement
{
    public int X { get; set; }
    public int Y { get; set; }

    // En instansmetod som använder objektets egna koordinater (this)
    public double DistanceTo(LevelElement other)
    {
        return Math.Sqrt(Math.Pow(X - other.X, 2) + Math.Pow(Y - other.Y, 2));
    }
}

// Inuti spel-loopen blir anropet nu helt naturligt och objektorienterat:
double dist = player.DistanceTo(enemy);
```

I "Gör HELLRE"-exemplet skapar vi en hög sammanhållning genom att låta LevelElement själv äga matematiken för sitt avstånd. Ett objekt ska lösa sina egna problem med hjälp av sin egen data (this.X och this.Y). Koden blir otroligt mycket mer intuitiv att läsa eftersom vi ber om avståndet från spelaren till fienden.

Varför strävar vi efter Clean Code?

Tre sanningar om programvara:

- **Kod läses mycket oftare än den skrivs**
  Du skriver koden en gång, men du (eller dina kollegor) kommer att läsa den dussintals gånger under veckorna som följer för att hitta buggar eller lägga till funktioner. Skriv kod för människan som ska läsa den, inte bara för datorn som ska köra den.

- **Clean Code handlar om hållbarhet över tid**
  I början av ett projekt går det snabbt att bara "hacka ihop" något. Men om koden saknar struktur (low cohesion, tight coupling) blir det till slut så trassligt att varje ny funktion introducerar tre nya buggar. God arkitektur gör att utvecklingen kan fortsätta hålla hög fart, även när projektet växer.

- **"Ett ärligt löfte"**
  Clean Code innebär att metoder och klasser gör exakt det deras namn utlovar. Det ska inte finnas några dolda sidoeffekter, inga överraskningar och inga 100 rader långa block som man måste scrolla igenom i blindo.

> *"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."*
> **— Martin Fowler**